

Lernziele:

- Umgang mit Strings
- Wiederholung: Gemeinsamkeiten und Unterschiede von Arrays und Pointern

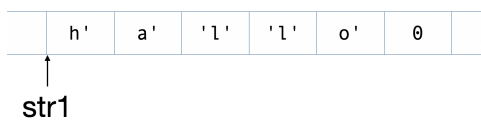
Schritt 1: Strings und Arrays (Wiederholung)

Ein String (Zeichenkette) ist in ABC (und C) ein Speicherbereich, der eine Sequenz von Zeichen enthält und mit einem Nullzeichen terminiert ist. Um einen String zu speichern, benötigt man also ein Array, und in dieses müssen nicht nur alle Zeichen, sondern auch das Nullzeichen passen.

Hier ein Beispiel für einen String "hallo", der in einem Array **str1** abgespeichert wird:

```
global str1: array[6] of char = {'h', 'a', 'l', 'l', 'o', 0};
```

Der String besteht hier aus 5 Zeichen und dem zusätzlichen Nullzeichen. Den Speicher kann man mit diesem Bild beschreiben:



Im List-Initializer wurden alle 6 Array-Elemente explizit initialisiert. Zur Erinnerung: Wenn man nicht alle Elemente explizit angibt, wird die Initializer-List vom Compiler mit Nullen aufgefüllt. Man hätte also auch äquivalent

```
global str1: array[6] of char = {'h', 'a', 'l', 'l', 'o'};
```

schreiben können. Wird ein Array in der Deklaration auch initialisiert, dann kann man die Dimension weglassen. Der Compiler setzt dann für uns die Dimension abhängig vom Initializer ein. Zum Beispiel wird bei

```
global str1: array[] of char = {'h', 'a', 'l', 'l', 'o', 0};
```

die Dimension auf 6 gesetzt. Und bei

```
global str1: array[] of char = {'h', 'a', 'l', 'l', 'o'};
```

auf 5, hier wird also im Array kein Nullzeichen abgespeichert. Als Initializer kann man bei Strings auch ein String-Literal verwenden, wie beispielsweise hier:

```
global str2: array[] of char = "hello";
```

Diese Initialisierung wird dann vom Compiler behandelt, als hätte man in einer Initializer-List alle Zeichen und zusätzlich das Nullzeichen aufgelistet. Es ist also äquivalent zu

```
global str2: array[] of char = {'h', 'e', 'l', 'l', 'o', 0};
```

Hätte man deshalb oben

```
global str2: array[5] of char = "hello";
```

geschrieben, dann hätte der Compiler festgestellt, dass die Dimension von **str2** zu klein ist.

Aufgabe

Schreibe folgendes Programm **ex1.abc**:

```
@ <stdio.h>

global str1: array[] of char = {'h', 'a', 'l', 'l', 'o'};
global str2: array[5] of char = "hello";

fn main()
{
    printf("str1 = %s\n", str1);
    printf("str2 = %s\n", str2);
}
```

1. Der Compiler wird sich beim Übersetzen beschweren, dass die Dimension von **str2** zu klein ist. Bestätige das zunächst. Ändere anschließend die Dimension auf 6.
2. Die Ausgabe auf meinem Rechner sieht wie folgt aus:

```
MCL:s22 lehn$ ./a.out
str1 = hallohello
str2 = hello
```

Wieso werden beim ersten Aufruf von printf beide Strings ausgegeben? Erkläre das mit einer Skizze für den Speicherinhalt.

3. Füge in der Funktion main folgende Anweisungen hinzu, um die Größe der Arrays auszugeben:

```
printf("sizeof(str1) = %zu\n", sizeof(str1));
printf("sizeof(str2) = %zu\n", sizeof(str2));
```

Ändere die Dimension von **str1** zu 42 und lass die Dimension bei **str2** weg.

4. Im Header **string.h** der Standard-Bibliothek ist die Funktion **strlen** deklariert, mit der man die Länge eines Strings bestimmen kann. Binde den Header mit

```
@ <string.h>
```

ein und füge die Anweisungen

```
printf("strlen(str1) = %zu\n", strlen(str1));
printf("strlen(str2) = %zu\n", strlen(str2));
```

in der Funktion main ein, um die String-Längen auszugeben. Ihr solltet jetzt feststellen, dass bei der Ausgabe der String-Länge das Nullzeichen nicht mitgezählt wird. Finde heraus, welche Stringlänge für **str1** ausgegeben wird, wenn die Dimension von **str1** auf 5 gesetzt wird oder die Dimension weggelassen wird.

Schritt 2: Strings und Pointer (Wiederholung)

Arrays und Pointer können ähnlich verwendet werden, obwohl es sich um unterschiedliche Datentypen handelt. Im folgenden Programm werden für **str1** und **str2** folgende Deklarationen verwendet:

```
global str1: array[] of char = "hallo";
global str2: -> char = str1;
```

Somit ist **str1** ein Array und **str2** ein Pointer. Die Initialisierung von **str2** ist dabei äquivalent zu:

```
global str2: -> char = &str1[0];
```

da der Name eines Arrays auch gleichzeitig die Adresse des ersten Array-Elements ist.

Aufgabe

1. Beschreibe die Speicherflächen für **str1** und **str2** im folgenden Programm:

```
@ <string.hdr>
@ <stdio.hdr>

global str1: array[] of char = "hallo";
global str2: -> char = &str1[0];

fn main()
{
  printf("sizeof(str1) = %zu\n", sizeof(str1));
  printf("sizeof(str2) = %zu\n", sizeof(str2));

  printf("strlen(str1) = %zu\n", strlen(str1));
  printf("strlen(str2) = %zu\n", strlen(str2));

  printf("str1 = %s\n", str1);
  printf("str2 = %s\n", str2);
}
```

2. Übersetze das Programm und führe es aus. Bestätige, dass **str1** und **str2** unterschiedliche Datentypen haben müssen, da die Größen der Variablen unterschiedlich sind.

Schritt 3 Call by Value

Bei einem Funktionsaufruf wird in ABC (und in C) immer eine Kopie des Arguments übergeben. In ABC gilt dies auch für Arrays (in C können Arrays aus historischen Gründen nicht als Parameter verwendet werden).

```
@ <stdio.hdr>

global str: array[42] of char = "hallo";

fn foo(str: array[42] of char)
{
  str[0] = 'X';
  printf("in foo: str = %s\n", str);
}

fn bar(str: -> char)
{
  str[0] = 'X';
  printf("in bar: str = %s\n", str);
}

fn main()
{
  printf("in main: str = %s\n", str);
  foo(str);
  printf("in main: str = %s\n", str);
  bar(str);
  printf("in main: str = %s\n", str);
}
```

Aufgabe

1. Erkläre deinem Nachbarn, warum nach dem Aufruf von **foo** die globale Variable **str** unverändert ist, aber nicht nach dem Aufruf von **bar**.
2. Erkläre deinem Nachbarn, warum ein Aufruf von **foo** "teurer" ist als der von **bar**.

Schritt 4 String-Funktionen

Einer Funktion wird in der Regel nicht das ganze String-Array übergeben, sondern nur ein Zeiger auf das erste Zeichen. Das hat folgende Vorteile:

1. Die Übergabe ist "billiger", da man nur den Zeiger (in der Regel 8 Bytes) übergibt und nicht eine komplette Kopie des Arrays.
2. Die Funktion kann den String verändern, falls das Sinn und Zweck der Funktion ist. Soll die Funktion den String nicht verändern, übergibt man einen Zeiger auf ein "readonly" Zeichen (siehe unten).
3. Würde man ein Array übergeben, dann muss die Dimension des Arrays in der Funktion explizit angegeben werden. Das ist aber nicht praktikabel, da Strings unterschiedliche Längen haben können und man sich deshalb auf eine maximale Länge (eine "worst case Länge") festlegen müsste.

In folgendem Programm wird eine Funktion `my_strlen` implementiert, die die Länge eines Strings bestimmen soll. Hier wird aber eben nicht ein Zeiger, sondern ein ganzes Array übergeben, was beim Übersetzen zu Problemen führen wird:

```
@ <stdio.h>

type string: array[42] of char;

fn my_strlen(str: string): size_t
{
    local len: size_t = 0;
    while (str[len]) {
        ++len;
    }
    return len;
}

fn main()
{
    printf("%zu\n", my_strlen("hallo"));
    printf("%zu\n", my_strlen("hi"));
    printf("%zu\n", my_strlen(""));
}
```

Aufgabe

1. Versuche das Programm in der jetzigen Form zu übersetzen. Dies sollte zu einem Fehler der Art:

```
ex4.abc:14.31-14.38: error: can not convert an expression of type 'array [6] of u8' to
type 'array [42] of char'
```

führen. Um dieses Problem zu beheben, ändere die Aufrufe so ab:

```
printf("%zu\n", my_strlen((string)"hallo"));
```

Damit werden die Aufrufe im Allgemeinen noch teurer: Erst wird durch den expliziten Cast ein temporäres Array angelegt, in das die Zeichenkette kopiert wird, und dann wird eine Kopie von diesem temporären Array übergeben.

2. Erkläre deinem Nachbarn, wie die Stringlänge bestimmt wird.
3. Entferne die Typvereinbarung für `string` und ändere die Deklaration der Funktion zu:

```
fn my_strlen(str: -> readonly char): size_t
```

Passe außerdem die Funktionsaufrufe entsprechend an, indem du den expliziten Cast wieder entfernst. Bestätige, dass die Implementierung nicht angepasst wird.

Schritt 5 Strings kopieren

Im Header **string.hdr** ist die Funktion **strcpy** wie folgt deklariert:

```
extern fn strcpy(dst: -> char, src: -> readonly char): -> char;
```

Mit der Funktion kann ein String kopiert werden:

- Der Parameter **src** (für Source) ist ein Zeiger auf das erste Zeichen eines Strings, der kopiert wird.
- Der Parameter **dst** (für Destination) ist ein Zeiger auf das erste Zeichen eines Strings (Ziel-String), der mit der Kopie überschrieben wird.
- Die Funktion gibt einen Zeiger auf das erste Zeichen des überschriebenen Strings zurück, also einfach den Wert von **dst**.

Die Funktion überprüft nicht, ob im Ziel-String ausreichend Platz ist, um dort eine komplette Kopie des Strings abzuspeichern. Das kann die Funktion auch gar nicht überprüfen, da sie nur Zeiger erhält, aber keine Information über die Größe der Arrays am Ende dieser Zeiger. Die Funktion kopiert einfach ein Zeichen nach dem anderen, bis das Nullzeichen kopiert wurde.

```
@ <string.hdr>
@ <stdio.hdr>

/*
// TODO
fn my_strcpy(dest: -> char, src: -> readonly char): -> char
{
}
*/

fn main()
{
  local str: array[6] of char = "1234";

  strcpy(str, "hallo");
  printf("%s\n", str);
}
```

Aufgabe

1. Übersetze das Programm und führe es aus.
2. Ändert man die Dimension von **str** auf 5, dann führt dies zu undefiniertem Verhalten. Bei mir erhalte ich einen Segmentation Fault, wenn ich ohne Optimierungen übersetze. Wenn ich mit **O3** übersetze, scheint aber alles problemlos zu funktionieren. Erkläre den Grund für das undefinierte Verhalten.
3. Implementiere deine eigene Funktion **my_strcpy**, um Strings zu kopieren. Ändere dazu auch den Aufruf von **strcpy** in der Funktion **main** zu einem Aufruf von **my_strcpy**. Stelle sicher, dass deine Funktion das Nullzeichen mitkopiert. Um dies im Test zu erkennen ändere die Deklaration von str zu


```
local str: array[] of char = "123456789";
```
4. Baue absichtlich einen Fehler ein: Ändere **my_strcpy** so, dass das Nullzeichen nicht kopiert wird. Beim Test sollte dies dadurch erkennbar sein, dass ein Teil des alten String sichtbar ist.

Schritt 6 Strings vergleichen

Im Header **string.hdr** ist die Funktion **strcmp** wie folgt deklariert:

```
extern fn strcmp(s1: -> readonly char, s2: -> readonly char): int;
```

Mit der Funktion können zwei Strings **s1** und **s2** verglichen werden:

- Sind beide Strings gleich, d.h. haben die gleiche Länge und stimmen alle Zeichen überein, dann gibt die Funktion 0 zurück.
- Sind die Strings verschieden und kommt **s1** lexikographisch vor **s2**, gibt die Funktion einen negativen Wert zurück.
- Sind die Strings verschieden und kommt **s1** lexikographisch nach **s2**, gibt die Funktion einen positiven Wert zurück.

Folgendes Programm zeigt die Verwendung von **strcmp** und eine mögliche eigene Implementierung:

```
@ <string.hdr>
@ <stdio.hdr>

fn my_strcmp(s1: -> readonly char, s2: -> readonly char): int
{
  for (; *s1 && *s1 == *s2; ++s1, ++s2) {
  }
  return (int)*s1 - (int)*s2;
}

fn main()
{
  local str1: array[] of char = "abcX";
  local str2: array[] of char = "abcx";

  printf("strcmp(str1, str2) = %d\n", strcmp(str1, str2));
  printf("my_strcmp(str1, str2) = %d\n", my_strcmp(str1, str2));

  if (!strcmp(str1, str2)) {
    printf("strings are equal\n");
  } else {
    printf("strings are different\n");
  }
}
```

Aufgabe

Teste das Programm für unterschiedliche Strings. Verwende hier auch Strings mit unterschiedlichen Längen. Erkläre deinem Nachbarn (oder mir):

1. Wieso der folgende Casts im Return-Statement notwendig ist:

```
return (int)*s1 - (int)*s2;
```

Bemerkung: Einfach mal die Casts weglassen, beobachten was die Funktion dann zurückgibt und letztlich Schlussfolgern was dafür der Grund sein muss.

2. Wie die Funktion **my_strcmp** funktioniert.